

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 1 ~u

```
    this.clampBasket();
  });

  this.updateStats();
  this.loop = setInterval(() => this.tick(), 16);
},

clampBasket() {
  this.basket.x = Math.max(0, Math.min(this.width - this.basket.w, this.basket.x));
},

tick() {
  if (!this.running) return;
  this.frameCount++;

  // Spawn items
  if (this.frameCount % 40 === 0) {
    const isBomb = Math.random() < 0.15;
    const emoji = isBomb ? 'ð=Û£' : ['ð<ßN', 'ð<ßJ', 'ð<ßK', 'ð<ßG', 'ð<ßS', 'ð<ßQ', 'ð<ßR', 'ð>Ý']
    '[Math.floor(Math.random() * 8)];
    this.items.push({
      x: Math.random() * (this.width - 30),
      y: -30,
      r: 18,
      emoji,
      isBomb,
      vy: this.speed + Math.random()
    });
  }

  // Increase speed over time
  if (this.frameCount % 500 === 0) this.speed += 0.3;

  // Move items
  for (const item of this.items) item.y += item.vy;

  // Check catch/miss
  this.items = this.items.filter(item => {
    if (item.y - item.r > this.height) {
      if (!item.isBomb) { this.lives--; this.updateStats(); }
      if (this.lives <= 0) { this.gameOver(); return false; }
    }
  });
}
```

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 33 ~u

```
        return false;
    }
    // Catch
    if (item.y + item.r > this.height - this.basket.h - 5 &&
        item.x > this.basket.x && item.x < this.basket.x + this.basket.w)
    {
        if (item.isBomb) {
            this.lives--;
            this.updateStats();
            if (this.lives <= 0) { this.gameOver(); return false; }
        } else {
            this.score += 10;
            this.updateStats();
        }
        return false;
    }
    return true;
});

this.draw();
},

draw() {
    const c = this.ctx;
    c.clearRect(0, 0, this.width, this.height);
    c.fillStyle = '#1a1a3e';
    c.fillRect(0, 0, this.width, this.height);

    // Items
    for (const item of this.items) {
        c.font = '28px sans-serif';
        c.textAlign = 'center';
        c.fillText(item.emoji, item.x, item.y);
    }

    // Basket
    c.fillStyle = '#a29bfe';
    c.fillRect(this.basket.x, this.height - this.basket.h, this.basket.w, this.basket.h);
    c.fillStyle = '#6c5ce7';
    c.fillRect(this.basket.x + 5, this.height - this.basket.h + 2, this.basket.w - 10, this.basket.h - 4);

    // Lives
    c.font = '14px sans-serif';
    c.textAlign = 'left';
    c.fillStyle = '#fff';
    c.fillText('dp '.repeat(this.lives), 8, 22);
}
```

```
=====
vÊfz\ n8b T -Æ^sSð|û~ß V3.2                {, 3 ~u
=====
```

```
    },

    updateStats() {
      if (this.statsCb) {
        this.statsCb({ '_-R ': this.score, 'u T}': this.lives });
      }
    }
  },

  n" id="replayBtn">Q•geN k!</button>";
  this.container.style.position = 'relative';
  this.container.appendChild(div);
  document.getElementById('replayBtn').addEventListener('click', () => {
    this.destroy();
    this.init(this.container, this.statsCb);
  });
},

destroy() {
  this.running = false;
  clearInterval(this.loop);
}
};
```

```
===== js/games/puzzle.js =====
```

```
/**
 * Sliding Puzzle (15-Puzzle / nÑwwbüVp)
 * @author ch2077
 * @version 1.1.0
 * @date 2025-02-15 R Qe• ^•epR$[šSi%ã`'
 * @date 2025-02-20 XžR shuffleeöV„Si%ãh!šŒÿ -2kbu b k{\@
 *
 * 4x4SAN"büVpÿ ep[W1-15+zz0M
 * p¹QÜN zz0Mvø•»v„nÑwwyûR`ÿ vôR0c'R N:1-15~z^•
 * -•g:bsNqeöu(• ^•[üepR$e-Si%ã`'ÿ OÝ<ÁN O u b eà%ãvø-b
 */
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 4 ~u

```
*/
const PuzzleGame = {
  size: 4,
  tiles: [],
  emptyIdx: 15,
  moves: 0,
  container: null,
  statsCb: null,
  gridEl: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.moves = 0;
  }
};
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 35 ~u

```

    this.tiles = Array.from({ length: this.size * this.size - 1 }, (_, i) => i + 1);
    this.tiles.push(0); // 0 = empty
    this.emptyIdx = this.tiles.indexOf(0);

    // Shuffle by making random valid moves
    for (let i = 0; i < 200; i++) {
      const neighbors = this.getNeighbors(this.emptyIdx);
      const pick = neighbors[Math.floor(Math.random() * neighbors.length)];
      this.swap(this.emptyIdx, pick);
      this.emptyIdx = pick;
    }

    this.render();
    this.updateStats();
  },

  getNeighbors(idx) {
    const r = Math.floor(idx / this.size);
    const c = idx % this.size;
    const result = [];
    if (r > 0) result.push(idx - this.size);
    if (r < this.size - 1) result.push(idx + this.size);
    if (c > 0) result.push(idx - 1);
    if (c < this.size - 1) result.push(idx + 1);
    return result;
  },
};
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 5 ~u

```
swap(a, b) {
  [this.tiles[a], this.tiles[b]] = [this.tiles[b], this.tiles[a]];
},

render() {
  this.container.innerHTML = '<div class="puzzle-grid"></div>';
  this.gridEl = this.container.querySelector('.puzzle-grid');

  this.tiles.forEach((val, i) => {
    const tile = document.createElement('div');
    tile.className = 'puzzle-tile' + (val === 0 ? ' empty' : '');
    tile.textContent = val || '';
    tile.addEventListener('click', () => this.move(i));
    this.gridEl.appendChild(tile);
  });
},

move(idx) {
  if (this.tiles[idx] === 0) return;
  const neighbors = this.getNeighbors(this.emptyIdx);
  if (!neighbors.includes(idx)) return;

  this.swap(idx, this.emptyIdx);
}
```

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 36 ~u

```
this.emptyIdx = idx;
this.moves++;
this.updateStats();
this.refreshDOM();

if (this.checkWin()) {
  this.showWin();
}
},

refreshDOM() {
  const tileEls = this.gridEl.querySelectorAll('.puzzle-tile');
  this.tiles.forEach((val, i) => {
    tileEls[i].textContent = val || '';
    tileEls[i].className = 'puzzle-tile' + (val === 0 ? ' empty' : '');
  });
},

checkWin() {
}
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 6 ~u

```
    for (let i = 0; i < this.tiles.length - 1; i++) {
      if (this.tiles[i] !== i + 1) return false;
    }
    return true;
  },

  updateStats() {
    if (this.statsCb) {
      this.statsCb({ 'keep': this.moves });
    }
  },

  showWin() {
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = `
      <h3>ð<ß% büVp[Æb ÿ </h3>
      <p>u(N† ${this.moves} ke</p>
      <button class="btn" id="replayBtn">Q•geN \@</button>
    `;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.init(this.container, this.statsCb);
    });
  },

  destroy() {}
};
```

===== js/games/tictactoe.js =====

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 37 ~u

```
/**
 * Tic-Tac-Toe (N•[WhĖ)
 * @author ch2077
 * @version 1.0.3
 * @date 2025-01-25 R rH
 * @date 2025-02-18 Aİe9u(minimaxÿ NÎ-•g:„=[PSG-ŞN:N Sİb €Ü
 *
 * 3x3N•[WhĖÿ s©[¶XQHbKvs AI(0)
 * AİO•u(minimax• _Rd }"g 0 %ăÿ OÝ<Á•ó\ ^s\@
 * ¸°_U€Ü• ^s~ß<ıÿ wins/losses/drawsÿ
```

```
*/
const TicTacToeGame = {
  board: [],
  player: 'X',
  ai: 'O',
  turn: 'X',
  wins: 0,
  losses: 0,
  draws: 0,
  container: null,
  statsCb: null,
  gridEl: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.board = Array(9).fill('');
    this.turn = 'X';
    this.render();
    this.updateStats();
  },

  render() {
    this.container.innerHTML = `
      <div class="tictactoe-grid"></div>
      <div class="instructions" style="margin-top:16px">O`bg Xÿ AI bg O</div>
    `;
    this.gridEl = this.container.querySelector('.tictactoe-grid');
    this.gridEl.style.cssText = 'display:grid;grid-template-columns:repeat(
3,80px);grid-template-rows:repeat(3,80px);gap:4px;';

    for (let i = 0; i < 9; i++) {
      const cell = document.createElement('div');
      cell.className = 'ttt-cell';
      cell.style.cssText = 'background:var(--surface2);border-radius:8px;di
splay:flex;align-items:center;justify-content:center;font-size:36px;font-we
ight:700;cursor:pointer;transition:background 0.15s;';
      cell.dataset.idx = i;
      cell.textContent = this.board[i];
      cell.addEventListener('click', () => this.playerMove(i, cell));
      this.gridEl.appendChild(cell);
    }
  }
}
```

```
},

playerMove(idx, cell) {
  if (this.board[idx] !== '' || this.turn !== 'X') return;
  this.board[idx] = 'X';
  cell.textContent = 'X';
  cell.style.color = '#54a0ff';

  if (this.checkWin('X')) {
    this.wins++;
    this.updateStats();
    this.endGame('O`•bN†ÿ ');
    return;
  }
  if (this.board.every(c => c !== '')) {
    this.draws++;
    this.updateStats();
    this.endGame('^s\@ÿ ');
    return;
  }

  this.turn = 'O';
  setTimeout(() => this.aiMove(), 300);
},

aiMove() {
  const move = this.getBestMove();
  this.board[move] = 'O';
  const cell = this.gridEl.querySelectorAll('.ttt-cell')[move];
  cell.textContent = 'O';
  cell.style.color = '#ff6b6b';

  if (this.checkWin('O')) {
    this.losses++;
    this.updateStats();
    this.endGame('AI •bN†ÿ ');
    return;
  }
  if (this.board.every(c => c !== '')) {
    this.draws++;
    this.updateStats();
    this.endGame('^s\@ÿ ');
    return;
  }
  this.turn = 'X';
},

getBestMove() {
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 9 ~u

```
// Minimax
let best = -Infinity;
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 39 ~u

```
let move = 0;
for (let i = 0; i < 9; i++) {
  if (this.board[i] === '') {
    this.board[i] = 'O';
    const score = this.minimax(0, false);
    this.board[i] = '';
    if (score > best) { best = score; move = i; }
  }
}
return move;
},

minimax(depth, isMax) {
  if (this.checkWin('O')) return 10 - depth;
  if (this.checkWin('X')) return depth - 10;
  if (this.board.every(c => c !== '')) return 0;

  if (isMax) {
    let best = -Infinity;
    for (let i = 0; i < 9; i++) {
      if (this.board[i] === '') {
        this.board[i] = 'O';
        best = Math.max(best, this.minimax(depth + 1, false));
        this.board[i] = '';
      }
    }
    return best;
  } else {
    let best = Infinity;
    for (let i = 0; i < 9; i++) {
      if (this.board[i] === '') {
        this.board[i] = 'X';
        best = Math.min(best, this.minimax(depth + 1, true));
        this.board[i] = '';
      }
    }
    return best;
  }
},
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2 {, 10 ~u

=====

```
checkWin(p) {
  const wins = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[
2,4,6]];
  return wins.some(w => w.every(i => this.board[i] === p));
},

updateStats() {
  if (this.statsCb) {
    this.statsCb({ '€Ü': this.wins, '• ': this.losses, '^s': this.draws });
  }
},
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2 {, 40 ~u

=====

```
endGame(msg) {
  this.gridEl.querySelectorAll('.ttt-cell').forEach(c => c.style.cursor =
'default');
  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  div.innerHTML = `<h3>${msg}</h3><button class="btn" id="replayBtn">Q•geN \@
</button>`;
  this.container.style.position = 'relative';
  this.container.appendChild(div);
  document.getElementById('replayBtn').addEventListener('click', () => {
    this.init(this.container, this.statsCb);
  });
},

destroy() {}
};
```

===== js/games/simon.js =====

```
/**
 * Simon Says (◁°_Æpo)
 * @author ch2077
 * @version 1.0.2
 * @date 2025-03-25 R rH
 * @date 2025-04-02 O S •êpÁ•ô-”ÿ R ••^•R eökİke•ô-”• Qİ
 *
 * ~İQx◁°_Æpon8b ÿ ◁°00poQI^•R ^vc ~z^•Y s°
 * kİ•n^•R ••^|+1ÿ g ••Sİeâ-Pc b ÿ • ◁İSs~Óg_
 * 4,rpoÿ ~¢„Ÿ-ÿžĂÿ %Æ%É+ÿ O*ÿ -óeHcDy:
 */
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 11 ~u

```
const SimonGame = {  
  sequence: [], playerId: 0,  
  showingSequence: false,  
  score: 0, highScore: 0,  
  container: null, statsCb: null,  
  colors: ['#ff6b6b', '#54a0ff', '#00d2a0', '#ffd93d'],  
  colorNames: ['~¢', '„Ÿ', '˘Ÿ', 'žÄ'],  
  soundEnabled: true,  
  
  init(container, statsCb) {  
    this.container = container;  
    this.statsCb = statsCb;  
    this.sequence = [];  
    this.playerId = 0;  
    this.showingSequence = false;  
    this.score = 0;  
    const saved = localStorage.getItem('simon_high');  
    this.highScore = saved ? parseInt(saved) : 0;  
    this.render();  
    setTimeout(() => this.nextRound(), 600);  
  },  
}
```

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 41 ~u

```
  
  render() {  
    this.container.innerHTML = `  
      <div style="text-align:center">  
        <div style="color:var(--text2);font-size:13px;margin-bottom:12px">  
00poQI`z^•Ÿ q6T 'İY p¹QûŸ </div>  
        <div class="simon-board">  
          ${this.colors.map((c, i) => `  
            <button class="simon-btn" data-idx="${i}" style="background:${c  
};aspect-ratio:1;border:none;border-radius:16px;cursor:pointer;transition:a  
ll 0.15s;box-shadow:0 0 0 rgba(0,0,0,0.3)"></button>  
          `).join('')}  
        </div>  
        <div id="simonStatus" style="margin-top:14px;font-weight:600;font-s  
ize:15px;color:var(--accent2)">QÆY ...</div>  
        <div style="color:var(--text2);font-size:12px;margin-top:8px">g šØR :  
${this.highScore}</div>  
      </div>`;  
  
    this.container.querySelectorAll('.simon-btn').forEach((btn, i) => {  
      btn.addEventListener('click', () => this.playerPress(i, btn));  
    })  
  }  
}
```

=====

vĖfz\ n8b T -Æ^sSđ|û~ß V3.2

=====

{, 12 ~u

```
    });ð
  },ð
ð
  nextRound() {ð
    this.showingSequence = true;ð
    this.playerIdx = 0;ð
    this.sequence.push(Math.floor(Math.random() * 4));ð
    document.getElementById('simonStatus').textContent = 'ˆ00~z^•...';ð
    this.updateStats();ð
    this.playSequence(0);ð
  },ð
ð
  playSequence(i) {ð
    if (i >= this.sequence.length) {ð
      this.showingSequence = false;ð
      document.getElementById('simonStatus').textContent = '•nR00`ÿ ';ð
      return;ð
    }ð
    const idx = this.sequence[i];ð
    const btn = this.container.querySelector(`.simon-btn[data-idx="${idx}"]`);ð
    this.lightUp(btn, idx);ð
    setTimeout(() => this.playSequence(i + 1), 500);ð
  },ð
ð
  lightUp(btn, idx) {ð
    btn.style.transform = 'scale(1.1)';ð
    btn.style.boxShadow = `0 0 24px ${this.colors[idx]}`;ð
    // Simple tone via oscillatorð
    if (this.soundEnabled) {ð
      try {ð
        const ctx = new (window.AudioContext || window.webkitAudioContext)();ð
        const osc = ctx.createOscillator();ð
        const gain = ctx.createGain();ð
        osc.connect(gain); gain.connect(ctx.destination);ð
      } catch(e) {}ð
    }ð
  }ð
}
```

vĖfz\ n8b T -Æ^sSđ|û~ß V3.2

{, 42 ~u

```
    osc.frequency.value = 300 + idx * 100;ð
    osc.type = 'sine';ð
    gain.gain.value = 0.1;ð
    osc.start(); osc.stop(ctx.currentTime + 0.15);ð
  } catch(e) {}ð
}ð
```

```

      setTimeout(() => {ð
        btn.style.transform = 'scale(1)';ð
        btn.style.boxShadow = '0 0 0 rgba(0,0,0,0.3)';ð
      }, 300);ð
    },ð
  ð
  playerPress(idx, btn) {ð
    if (this.showingSequence) return;ð
    this.lightUp(btn, idx);ð
  ð
    if (idx !== this.sequence[this.playerIdx]) {ð
      document.getElementById('simonStatus').innerHTML = '<span style="color:#ff6b6b">• <iÿ </span>';ð
      if (this.score > this.highScore) {ð
        this.highScore = this.score;ð
        localStorage.setItem('simon_high', this.highScore);ð
      }ð
      setTimeout(() => this.showGameOver(), 800);ð
      return;ð
    }ð
  ð
    this.playerIdx++;ð
    if (this.playerIdx >= this.sequence.length) {ð
      this.score++;ð
      document.getElementById('simonStatus').textContent = 'kcxnÿ N N •n...';ð
      setTimeout(() => this.nextRound(), 800);ð
    }ð
    this.updateStats();ð
  },ð
  ð
  updateStats() {ð
    if (this.statsCb) this.statsCb({ '_-R ': this.score, '^•R ••^|': this.sequence.length });ð
  },ð
  ð
  showGameOver() {ð
    const div = document.createElement('div');ð
    div.className = 'game-over-overlay';ð
    div.innerHTML = `<h3>n8b ~ôg_</h3><p>R ep: ${this.score} | g šØR : ${this.highScore}</p><button class="btn" id="replayBtn">Q•geN k!</button>`;ð
    this.container.style.position = 'relative';ð
    this.container.appendChild(div);ð
    document.getElementById('replayBtn').addEventListener('click', () => {ð
      this.init(this.container, this.statsCb);ð
    });ð
  },ð
  ð
  ð
  -----

```

=====

vĖfz\ n8b T -Æ^sSđ û~ß V3.2	{, 14 ~u
-----------------------------	----------

=====

vĖfz\ n8b T -Æ^sSđ û~ß V3.2	{, 43 ~u
-----------------------------	----------

```
destroy() {}  
};
```

===== js/games/reaction.js =====

```
/**  
 * Reaction Time (SÍ^"• ^!mK<đ)  
 * @author ch2077  
 * @version 1.0.3  
 * @date 2025-02-01 R rH  
 * @date 2025-03-08 R N†pre-clickhÂmKÿ -2kb~,R$cĐRMp¹Qû  
 *  
 * \0^US0~yT \=_ëp¹Qûÿ mK<đSÍ^"• ^!ÿ msÿ  
 * r¶` g:ÿ idle!'waiting(random delay)!'ready!'clicked  
 * <°_Ug •N5k!b ~é+g Osb ~é  
 */  
const ReactionGame = {  
  state: 'idle', // idle, waiting, ready, clicked  
  startTime: 0,  
  timeout: null,  
  best: Infinity,  
  results: [],  
  container: null,  
  statsCb: null,  
  
  init(container, statsCb) {  
    this.container = container;  
    this.statsCb = statsCb;  
    this.state = 'idle';  
    this.results = [];  
    this.render();  
    this.updateStats();  
  },  
  
  render() {  
    this.container.innerHTML = `  
      <div style="text-align:center">  
        <div class="reaction-zone" id="reactionZone"  
          style="width:200px;height:200px;border-radius:50%;margin:20px aut  
o;display:flex;align-items:center;justify-content:center;cursor:pointer;tra  
nsition:background 0.1s,user-select:none;font-size:16px;font-weight:700">  
          <span id="zoneText">p¹b _ YĖ</span>  
        </div>
```

```
<div id="reactionResults" style="margin-top:16px"></div>
</div>
`;

const zone = document.getElementById('reactionZone');
this.updateZoneStyle(zone, 'idle');

zone.addEventListener('click', () => {
  if (this.state === 'idle' || this.state === 'clicked') {
```

```
    this.startCountdown();
  } else if (this.state === 'waiting') {
    this.tooEarly();
  } else if (this.state === 'ready') {
    this.recordReaction();
  }
});
},

updateZoneStyle(zone, state) {
  if (state === 'idle') {
    zone.style.background = 'var(--accent)';
    zone.style.color = '#fff';
    document.getElementById('zoneText').textContent = 'p¹b _ YË';
  } else if (state === 'waiting') {
    zone.style.background = '#ff6b6b';
    zone.style.color = '#fff';
    document.getElementById('zoneText').textContent = '{I_...~ÿ,r...';
  } else if (state === 'ready') {
    zone.style.background = '#00d2a0';
    zone.style.color = '#fff';
    document.getElementById('zoneText').textContent = '_ëc ÿ ';
  } else if (state === 'clicked') {
    zone.style.background = 'var(--accent)';
    zone.style.color = '#fff';
    document.getElementById('zoneText').textContent = 'Q•<ÖN k!';
  }
},

startCountdown() {
  this.state = 'waiting';
  const zone = document.getElementById('reactionZone');
  this.updateZoneStyle(zone, 'waiting');
```

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 16 ~u

```
const delay = 1000 + Math.random() * 3000;
clearTimeout(this.timeout);
this.timeout = setTimeout(() => {
  this.state = 'ready';
  this.startTime = Date.now();
  this.updateZoneStyle(zone, 'ready');
}, delay);
},
tooEarly() {
  this.state = 'clicked';
  clearTimeout(this.timeout);
  const zone = document.getElementById('reactionZone');
  zone.style.background = '#ffa726';
  document.getElementById('zoneText').textContent = 'Y*eéN†ÿ {I~ÿ,rQ•c ';
  setTimeout(() => {
```

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 45 ~u

```
    this.state = 'idle';
    this.updateZoneStyle(zone, 'idle');
  }, 1500);
},
recordReaction() {
  this.state = 'clicked';
  const time = Date.now() - this.startTime;
  this.results.unshift(time);
  if (this.results.length > 5) this.results.pop();
  if (time < this.best) this.best = time;

  const zone = document.getElementById('reactionZone');
  document.getElementById('zoneText').textContent = `${time} ms`;
  this.updateStats();
  this.showResults();

  setTimeout(() => {
    this.state = 'idle';
    this.updateZoneStyle(zone, 'idle');
  }, 2000);
},
showResults() {
  const el = document.getElementById('reactionResults');
```

```

    el.innerHTML = `<p style="color:var(--text2);font-size:13px;margin-bott
om:8px">g •Ñ 5 k!ÿ </p>
    <div style="display:flex;gap:6px;justify-content:center;flex-wrap:wra
p">
        ${this.results.map((t, i) => `<span style="padding:4px 10px;border-
radius:20px;font-size:12px;font-weight:600;
        background:${i===0?'var(--accent)':'var(--surface2)'};color:${i==
=0?'#fff':'var(--text)'}"
        >${t}ms</span>`).join('')}
    </div>`;
},

updateStats() {
    if (this.statsCb) {
        this.statsCb({ 'g 0s': this.best === Infinity ? '--' : `${this.best}ms`
});
    }
},

destroy() {
    clearTimeout(this.timeout);
}
};
```

===== js/games/numberguess.js =====

```

/**
 * Number Guessing (ep[ws Æ )
 * @author ch2077
 * @version 1.0.2
```

```

* @date 2025-01-28 R rH
* @date 2025-02-08 f VôcÐy:e9N:R`` S:•ôf>y:
*
* s 1-100-•g:epÿ kİk!s [ÆcÐy:"Y'N†/\ N†"^v•)\ f Vô
* <¡k!~ß<¡ÿ cÐ0>'İ•n'İe°_ \@
*/
const NumberGuessGame = {
    target: 0,
    attempts: 0,
    range: [1, 100],
    container: null,
    statsCb: null,
```

```
init(container, statsCb) {
  this.container = container;
  this.statsCb = statsCb;
  this.target = Math.floor(Math.random() * 100) + 1;
  this.attempts = 0;
  this.range = [1, 100];
  this.render();
  this.updateStats();
},

render() {
  this.container.innerHTML = `
    <div style="text-align:center">
      <div style="font-size:60px;margin-bottom:12px">0>Ÿ </div>
      <p style="color:var(--text2);margin-bottom:8px">b `óN†N N* 1~100 NK•ôv„ep[W</p>
    <p style="color:var(--accent2);font-size:14px;margin-bottom:20px" i
d="rangeHint">f Vô: 1 ~ 100</p>
    <div style="display:flex;gap:8px;justify-content:center;margin-bottom:16px">
      <input type="number" id="guessInput" min="1" max="100" placeholder="•“Qeep[W"
        style="width:140px;padding:10px 14px;border-radius:8px;border:1
px solid var(--border);background:var(--surface2);color:var(--text);font-si
ze:16px;text-align:center">
      <button class="btn" id="guessBtn">s Ÿ </button>
    </div>
    <div id="resultHint" style="font-size:15px;min-height:24px"></div>
    <div id="history" style="margin-top:12px;display:flex;flex-wrap:wrap;wra
p;gap:6px;justify-content:center"></div>
  `;

  const input = document.getElementById('guessInput');
  document.getElementById('guessBtn').addEventListener('click', () => thi
s.guess());
  input.addEventListener('keydown', (e) => {
    if (e.key === 'Enter') this.guess();
  });
  input.focus();
},

guess() {
  const input = document.getElementById('guessInput');
  const val = parseInt(input.value);
```

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

{, 19 ~u

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

{, 47 ~u

```
if (isNaN(val) || val < 1 || val > 100) {
  document.getElementById('resultHint').innerHTML = '<span style="color
: #ff6b6b"><÷•"Qe 1-100 v„ep[W</span>';
  return;
}

this.attempts++;
this.updateStats();

const hint = document.getElementById('resultHint');
const history = document.getElementById('history');
const badge = document.createElement('span');
badge.style.cssText = 'padding:4px 10px;border-radius:20px;font-size:12
px;font-weight:600;';

if (val === this.target) {
  hint.innerHTML = `<span style="color:#00d2a0;font-size:18px">0<ß% s [ùN†ÿ ep
[W\1f/ ${this.target}</span>`;
  badge.style.background = 'var(--green)';
  badge.style.color = '#fff';
  badge.textContent = val + ' ' ' ';
  history.appendChild(badge);
  this.showWin();
  return;
}

if (val < this.target) {
  this.range[0] = Math.max(this.range[0], val + 1);
  hint.innerHTML = `<span style="color:#ffa726">0=ÛÉ Y*\ N†ÿ Q•Y'N p¹</span>`;
  badge.style.background = '#3a2a10';
  badge.style.color = '#ffa726';
} else {
  this.range[1] = Math.min(this.range[1], val - 1);
  hint.innerHTML = `<span style="color:#54a0ff">0=ÛÉ Y*Y'N†ÿ Q•\ N p¹</span>`;
  badge.style.background = '#1a2a3a';
  badge.style.color = '#54a0ff';
}
badge.textContent = val;
history.appendChild(badge);

document.getElementById('rangeHint').textContent = `f Vð: ${this.range[0]
} ~ ${this.range[1]}`;
input.value = '';
input.focus();
```

```
=====
vÊfz\ n8b T -Æ^sSð|û-ß V3.2                {, 20 ~u
=====
```

```
    },

    updateStats() {
      if (this.statsCb) {
        this.statsCb({ 's mKk!ep': this.attempts });
      }
    },

    showWin() {
      document.getElementById('guessBtn').disabled = true;
    }
  },

  {, 48 ~u

  document.getElementById('guessInput').disabled = true;
  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  div.innerHTML = `<h3>ð<ß% s [ùN†ÿ </h3><p>u(N† ${this.attempts} k!</p><button cl
ass="btn" id="replayBtn">Q•geN \@</button>`;
  this.container.style.position = 'relative';
  this.container.appendChild(div);
  document.getElementById('replayBtn').addEventListener('click', () => {
    this.init(this.container, this.statsCb);
  });
},

  destroy() {}
};
```

```
===== js/games/colormatch.js =====
```

```
/**
 * Color Match / Stroop Test (~æ,rS9'M)
 * @author ch2077
 * @version 1.0.4
 * @date 2025-02-12 R rHÿ WÚNŽStroopeH^"ÿ
 * @date 2025-03-10 30y0-Peö+•pQûR b
 *
 * R$e-e‡[W~æ,rƒ/T&N e‡[WQ…[¹N •ôÿ 30y0-Peö
 * StroopeH^"ÿ e‡[W<ô"-¢"0Ff>y:„Ÿ,r!‘SÍ^"S0ab
 * •pQû(Streak)<¡k!ÿ g š0•pQû<°_U0Y[X
 */
const ColorMatchGame = {
  colorName: '',
  displayColor: '',
  score: 0,
}
```

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

{, 21 ~u

```
streak: 0,
bestStreak: 0,
timeLeft: 30,
running: false,
timer: null,
container: null,
statsCb: null,
colors: ['~¢', '„Ÿ', '~Ÿ', 'žÄ', 'jY', '}]+'],
colorMap: { '~¢': '#ff6b6b', '„Ÿ': '#54a0ff', '~Ÿ': '#00d2a0', 'žÄ': '#ffd93d',
'jY': '#ffa726', '}]+'': '#a29bfe' },
```

```
init(container, statsCb) {
  this.container = container;
  this.statsCb = statsCb;
  this.score = 0;
  this.streak = 0;
  this.bestStreak = 0;
  this.timeLeft = 30;
  this.running = true;
  this.render();
}
```

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

{, 49 ~u

```
this.nextRound();
this.startTimer();
this.updateStats();
},

render() {
  this.container.innerHTML = `
    <div style="text-align:center">
      <div style="color:var(--text2);font-size:14px;margin-bottom:8px">e‡[W
~æ,rN T yðf/T&N •ðŸ </div>
      <div id="colorDisplay" style="font-size:48px;font-weight:800;min-height:80px;
display:flex;align-items:center;justify-content:center;transition:all 0.15s"></div>
      <div style="display:flex;gap:12px;justify-content:center;margin-top:20px">
        <button class="btn" id="matchYes" style="font-size:18px;padding:14px 36px;
background:var(--green)">' N •ð</button>
        <button class="btn" id="matchNo" style="font-size:18px;padding:14px 36px;
background:var(--red)">'L N N •ð</button>
      </div>
      <div id="matchResult" style="margin-top:16px;min-height:28px;font-size:15px;
font-weight:600"></div>
  `
}
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 22 ~u

```
    <div style="margin-top:12px;color:var(--text2);font-size:13px">
      •pQû: <span id="streakDisplay" style="color:var(--accent2);font-weight:700">0</span>
    </div>
  </div>
`;
```

```
    document.getElementById('matchYes').addEventListener('click', () => this.s.answer(true));
    document.getElementById('matchNo').addEventListener('click', () => this.s.answer(false));
  },
```

```
  nextRound() {
    if (!this.running) return;

    // Randomly choose whether color matches name
    const match = Math.random() < 0.4; // 40% chance of match
    this.colorName = this.colors[Math.floor(Math.random() * this.colors.length)];
```

```
    if (match) {
      this.displayColor = this.colorMap[this.colorName];
    } else {
      // Pick a different color
      const others = this.colors.filter(c => c !== this.colorName);
      const pick = others[Math.floor(Math.random() * others.length)];
      this.displayColor = this.colorMap[pick];
    }
  }
```

```
  const display = document.getElementById('colorDisplay');
  if (display) {
    display.textContent = this.colorName;
    display.style.color = this.displayColor;
  }
},
```

```
  answer(userSaysMatch) {
    if (!this.running) return;
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 50 ~u

```
    const actualMatch = this.displayColor === this.colorMap[this.colorName];
;
    const correct = (userSaysMatch === actualMatch);
```

```
const resultEl = document.getElementById('matchResult');
if (correct) {
  this.score += 10 + this.streak * 2;
  this.streak++;
  if (this.streak > this.bestStreak) this.bestStreak = this.streak;
  resultEl.innerHTML = '<span style="color:#00d2a0">kcxny </span>';
} else {
  this.streak = 0;
  resultEl.innerHTML = '<span style="color:#ff6b6b">• Nty </span>';
}

document.getElementById('streakDisplay').textContent = this.streak;
this.updateStats();
this.nextRound();
},

startTimer() {
  this.timer = setInterval(() => {
    this.timeLeft--;
    this.updateStats();
    if (this.timeLeft <= 0) {
      this.gameOver();
    }
  }, 1000);
},

updateStats() {
  if (this.statsCb) {
    this.statsCb({ '_-R ': this.score, '•pQû': this.streak, 'eö•ô': `${this.timeLeft}s` });
  }
},

gameOver() {
  this.running = false;
  clearInterval(this.timer);
  if (this.streak > this.bestStreak) this.bestStreak = this.streak;

  const div = document.createElement('div');
  div.className = 'game-over-overlay';
  div.innerHTML = `
    <h3>eö•ôR0y </h3>
    <p>_-R : ${this.score} | g 0s•pQû: ${this.bestStreak}</p>
    <button class="btn" id="replayBtn">Q•geN k!</button>
  `;
  this.container.style.position = 'relative';
  this.container.appendChild(div);
}
```

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 24 ~u

document.getElementById('replayBtn').addEventListener('click', () => {

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 51 ~u

 this.destroy();
 this.init(this.container, this.statsCb);
 });
},

destroy() {
 this.running = false;
 clearInterval(this.timer);
}
};

===== css/style.css =====

/* === Reset & Base === */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
:root {
 --bg: #0f0f1a;
 --surface: #1a1a2e;
 --surface2: #252542;
 --border: #333355;
 --text: #e0e0f0;
 --text2: #8888aa;
 --accent: #6c5ce7;
 --accent2: #a29bfe;
 --green: #00d2a0;
 --red: #ff6b6b;
 --orange: #ffa726;
 --yellow: #ffd93d;
 --blue: #54a0ff;
 --radius: 12px;
 --shadow: 0 4px 24px rgba(0,0,0,0.3);
 --transition: 0.2s ease;
}
html, body {
 height: 100%;
 font-family: 'Segoe UI', system-ui, -apple-system, sans-serif;
 background: var(--bg);
 color: var(--text);
 overflow-x: hidden;
}

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 25 ~u

=====

```
}Đ
Đ
#app {Đ
  min-height: 100vh;Đ
  display: flex;Đ
  flex-direction: column;Đ
}Đ
Đ
/* === Header === */Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 52 ~u

=====

```
.app-header {Đ
  display: flex;Đ
  align-items: center;Đ
  justify-content: space-between;Đ
  padding: 16px 24px;Đ
  background: var(--surface);Đ
  border-bottom: 1px solid var(--border);Đ
  position: sticky;Đ
  top: 0;Đ
  z-index: 100;Đ
  backdrop-filter: blur(10px);Đ
}Đ
Đ
.header-left { display: flex; align-items: center; gap: 12px; }Đ
.logo { font-size: 28px; }Đ
.app-header h1 { font-size: 20px; font-weight: 700; color: var(--text); }Đ
Đ
.btn-back {Đ
  background: var(--surface2);Đ
  color: var(--text);Đ
  border: 1px solid var(--border);Đ
  padding: 8px 16px;Đ
  border-radius: 8px;Đ
  cursor: pointer;Đ
  font-size: 14px;Đ
  transition: var(--transition);Đ
}Đ
.btn-back:hover { background: var(--border); }Đ
Đ
/* === Home Page === */Đ
.home-page { padding: 40px 24px; max-width: 1000px; margin: 0 auto; width:
100%; }Đ
Đ
-----
```

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

{, 26 ~u

```
.hero {ð
  text-align: center;ð
  margin-bottom: 40px;ð
}ð
.hero h2 { font-size: 28px; margin-bottom: 8px; color: var(--accent2); }ð
.hero p { font-size: 16px; color: var(--text2); }ð
ð
.game-grid {ð
  display: grid;ð
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));ð
  gap: 20px;ð
}ð
ð
.game-card {ð
  background: var(--surface);ð
  border: 1px solid var(--border);ð
  border-radius: var(--radius);ð
  padding: 24px;ð
}
```

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

{, 53 ~u

```
  cursor: pointer;ð
  transition: transform var(--transition), box-shadow var(--transition), bo
rder-color var(--transition);ð
  display: flex;ð
  flex-direction: column;ð
  gap: 12px;ð
}ð
.game-card:hover {ð
  transform: translateY(-4px);ð
  box-shadow: var(--shadow);ð
  border-color: var(--accent);ð
}ð
.game-card .game-icon { font-size: 40px; }ð
.game-card .game-name { font-size: 18px; font-weight: 600; }ð
.game-card .game-desc { font-size: 13px; color: var(--text2); line-height:
1.5; }ð
.game-card .game-tags { display: flex; gap: 6px; flex-wrap: wrap; }ð
.game-card .tag {ð
  font-size: 11px;ð
  padding: 3px 10px;ð
  border-radius: 20px;ð
  background: var(--surface2);ð
  color: var(--text2);ð
  border: 1px solid var(--border);ð
}
```

```
}Đ
Đ
/* === Game Page === */Đ
.game-page { padding: 20px 24px 40px; max-width: 700px; margin: 0 auto; wid
th: 100%; }Đ
Đ
.game-header {Đ
  display: flex;Đ
  align-items: center;Đ
  justify-content: space-between;Đ
  flex-wrap: wrap;Đ
  gap: 12px;Đ
  margin-bottom: 20px;Đ
}Đ
.game-header h2 { font-size: 22px; color: var(--accent2); }Đ
Đ
.game-controls { display: flex; align-items: center; gap: 12px; flex-wrap:
wrap; }Đ
Đ
.game-stats { display: flex; gap: 16px; }Đ
Đ
.stat {Đ
  background: var(--surface2);Đ
  padding: 6px 14px;Đ
  border-radius: 8px;Đ
  font-size: 13px;Đ
  color: var(--text2);Đ
}Đ
.stat span { color: var(--text); font-weight: 600; }Đ
Đ
```

```
.btn {Đ
  background: var(--accent);Đ
  color: #fff;Đ
  border: none;Đ
  padding: 8px 18px;Đ
  border-radius: 8px;Đ
  cursor: pointer;Đ
  font-size: 14px;Đ
  font-weight: 600;Đ
  transition: var(--transition);Đ
}Đ
.btn:hover { opacity: 0.85; transform: scale(1.02); }Đ
```

```
.btn:active { transform: scale(0.98); }ð
.btn.small { padding: 5px 12px; font-size: 12px; }ð
ð
.game-area {ð
  background: var(--surface);ð
  border: 1px solid var(--border);ð
  border-radius: var(--radius);ð
  padding: 24px;ð
  min-height: 400px;ð
  display: flex;ð
  flex-direction: column;ð
  align-items: center;ð
  justify-content: center;ð
}ð
ð
/* === Footer === */ð
.app-footer {ð
  text-align: center;ð
  padding: 20px;ð
  color: var(--text2);ð
  font-size: 13px;ð
  border-top: 1px solid var(--border);ð
  margin-top: auto;ð
}ð
ð
/* === Memory Game === */ð
.memory-grid {ð
  display: grid;ð
  gap: 8px;ð
  justify-content: center;ð
  justify-items: stretch;ð
}ð
ð
.memory-card {ð
  width: 72px;ð
  height: 72px;ð
  background: var(--surface2);ð
  border: 2px solid var(--border);ð
```

```
border-radius: 10px;ð
cursor: pointer;ð
font-size: 32px;ð
display: flex;ð
```

```
align-items: center;Đ
justify-content: center;Đ
transition: transform 0.3s ease, background var(--transition);Đ
user-select: none;Đ
}Đ
.memory-card:hover { border-color: var(--accent); }Đ
.memory-card.flipped { background: var(--accent); border-color: var(--accen
t2); }Đ
.memory-card.matched { background: var(--green); border-color: var(--green)
; opacity: 0.7; }Đ
Đ
/* === 2048 === */Đ
.game-2048 { width: 100%; max-width: 400px; }Đ
.board-2048 {Đ
  display: grid;Đ
  grid-template-columns: repeat(4, 1fr);Đ
  grid-template-rows: repeat(4, 1fr);Đ
  gap: 8px;Đ
  aspect-ratio: 1;Đ
  background: var(--surface2);Đ
  padding: 8px;Đ
  border-radius: var(--radius);Đ
}Đ
Đ
.cell-2048 {Đ
  background: var(--bg);Đ
  border-radius: 8px;Đ
  display: flex;Đ
  align-items: center;Đ
  justify-content: center;Đ
  font-size: 22px;Đ
  font-weight: 700;Đ
  transition: all 0.1s ease;Đ
}Đ
Đ
/* === Minesweeper === */Đ
.mine-grid {Đ
  display: grid;Đ
  gap: 2px;Đ
  background: var(--surface2);Đ
  padding: 4px;Đ
  border-radius: var(--radius);Đ
}Đ
Đ
.mine-cell {Đ
  width: 32px;Đ
  height: 32px;Đ
  background: var(--bg);Đ
```

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

{, 30 ~u

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

{, 56 ~u

```
border: 1px solid var(--border);ð
border-radius: 4px;ð
display: flex;ð
align-items: center;ð
justify-content: center;ð
font-size: 13px;ð
font-weight: 700;ð
cursor: pointer;ð
user-select: none;ð
transition: background 0.1s;ð
}ð
.mine-cell:hover { background: #2a2a4a; }ð
.mine-cell.revealed { background: var(--surface2); }ð
.mine-cell.flagged { background: var(--orange); }ð
.mine-cell.mine { background: var(--red); }ð
ð
/* === Sliding Puzzle === */ð
.puzzle-grid {ð
  display: grid;ð
  grid-template-columns: repeat(4, 70px);ð
  grid-template-rows: repeat(4, 70px);ð
  gap: 4px;ð
  background: var(--surface2);ð
  padding: 8px;ð
  border-radius: var(--radius);ð
}ð
ð
.puzzle-tile {ð
  background: var(--accent);ð
  color: #fff;ð
  border-radius: 8px;ð
  display: flex;ð
  align-items: center;ð
  justify-content: center;ð
  font-size: 20px;ð
  font-weight: 700;ð
  cursor: pointer;ð
  transition: transform 0.15s ease, background var(--transition);ð
  user-select: none;ð
}ð
.puzzle-tile:hover { opacity: 0.85; }ð
.puzzle-tile.empty { background: transparent; cursor: default; }ð
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 31 ~u

ð

```
/* === Snake / Tetris Canvas === */ð
canvas.game-canvas {ð
  border-radius: 8px;ð
  display: block;ð
  background: var(--bg);ð
}ð
ð
```

=====

vÊfz\ n8b T -Æ^sSð|û-ß V3.2

=====

{, 57 ~u

```
.instructions {ð
  text-align: center;ð
  color: var(--text2);ð
  font-size: 13px;ð
  margin-top: 12px;ð
}ð
ð
/* === Game Over Overlay === */ð
.game-over-overlay {ð
  position: absolute;ð
  inset: 0;ð
  background: rgba(0,0,0,0.7);ð
  display: flex;ð
  flex-direction: column;ð
  align-items: center;ð
  justify-content: center;ð
  border-radius: var(--radius);ð
  gap: 16px;ð
}ð
.game-over-overlay h3 { font-size: 24px; }ð
ð
/* === Level Transition Overlay === */ð
.level-intro {ð
  position: absolute;ð
  inset: 0;ð
  display: flex;ð
  flex-direction: column;ð
  align-items: center;ð
  justify-content: center;ð
  background: rgba(0,0,0,0.82);ð
  border-radius: var(--radius);ð
  z-index: 20;ð
  animation: levelFade 1.5s ease forwards;ð
  pointer-events: none;ð
}
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 32 ~u

```
}Đ
.level-intro .level-num {Đ
  font-size: 42px;Đ
  font-weight: 700;Đ
  color: var(--accent2);Đ
  text-shadow: 0 0 20px rgba(108,92,231,0.5);Đ
}Đ
.level-intro .level-title {Đ
  font-size: 16px;Đ
  color: var(--text);Đ
  margin-top: 8px;Đ
  opacity: 0.8;Đ
}Đ
Đ
@keyframes levelFade {Đ
  0% { opacity: 1; }Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 58 ~u

```
  65% { opacity: 1; }Đ
  100% { opacity: 0; }Đ
}Đ
Đ
/* === Responsive === */Đ
@media (max-width: 600px) {Đ
  .app-header { padding: 12px 16px; }Đ
  .app-header h1 { font-size: 17px; }Đ
  .hero h2 { font-size: 22px; }Đ
  .home-page { padding: 24px 16px; }Đ
  .game-page { padding: 16px; }Đ
  .game-grid { grid-template-columns: 1fr; }Đ
  .game-area { padding: 16px; }Đ
Đ
  .memory-card { width: 58px; height: 58px; font-size: 24px; }Đ
Đ
  .puzzle-grid {Đ
    grid-template-columns: repeat(4, 60px);Đ
    grid-template-rows: repeat(4, 60px);Đ
  }Đ
  .puzzle-tile { font-size: 18px; }Đ
Đ
  .mine-cell { width: 28px; height: 28px; font-size: 11px; }Đ
Đ
  .cell-2048 { font-size: 18px; }Đ
}Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 33 ~u

```
Đ
/* === Tile colors for 2048 === */Đ
.tile-2 { background: #eee4da; color: #776e65; }Đ
.tile-4 { background: #ede0c8; color: #776e65; }Đ
.tile-8 { background: #f2b179; color: #f9f6f2; }Đ
.tile-16 { background: #f59563; color: #f9f6f2; }Đ
.tile-32 { background: #f67c5f; color: #f9f6f2; }Đ
.tile-64 { background: #f65e3b; color: #f9f6f2; }Đ
.tile-128 { background: #edcf72; color: #f9f6f2; }Đ
.tile-256 { background: #edcc61; color: #f9f6f2; }Đ
.tile-512 { background: #edc850; color: #f9f6f2; }Đ
.tile-1024 { background: #edc53f; color: #f9f6f2; }Đ
.tile-2048 { background: #edc22e; color: #f9f6f2; }Đ
Đ
/* Position relative for game area to support overlays */Đ
.game-area { position: relative; }Đ
Đ
/* === Delta Force Randomizer === */Đ
.delta-grid {Đ
  display: grid;Đ
  grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));Đ
  gap: 12px;Đ
  width: 100%;Đ
}Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û-ß V3.2

=====

{, 59 ~u

```
Đ
.delta-slot {Đ
  background: var(--surface2);Đ
  border: 2px solid var(--border);Đ
  border-radius: var(--radius);Đ
  padding: 12px;Đ
  text-align: center;Đ
  transition: border-color var(--transition), box-shadow var(--transition);
}Đ
Đ
.delta-photo {Đ
  width: 100%;Đ
  aspect-ratio: 4/3;Đ
  border-radius: 8px;Đ
  display: flex;Đ
  align-items: center;Đ
  justify-content: center;Đ
}Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û~ß V3.2

=====

{, 34 ~u

```
    transition: background 0.15s ease, transform 0.15s ease;Đ
    margin-bottom: 8px;Đ
    background: var(--bg);Đ
}Đ
Đ
.delta-photo-icon {Đ
    font-size: 36px;Đ
    filter: drop-shadow(0 2px 4px rgba(0,0,0,0.4));Đ
}Đ
Đ
.photo-done {Đ
    transform: scale(1.05);Đ
    box-shadow: 0 0 16px rgba(108,92,231,0.4);Đ
}Đ
Đ
.delta-label-sm {Đ
    font-size: 12px;Đ
    color: var(--text2);Đ
    margin-bottom: 4px;Đ
}Đ
Đ
.delta-display {Đ
    font-size: 14px;Đ
    font-weight: 700;Đ
    min-height: 20px;Đ
    transition: color 0.1s;Đ
}Đ
Đ
.delta-display.rolling {Đ
    animation: deltaShake 0.06s infinite;Đ
}Đ
Đ
.delta-display.done {Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û~ß V3.2

=====

{, 60 ~u

```
    animation: deltaPop 0.3s ease;Đ
}Đ
Đ
@keyframes deltaShake {Đ
    0% { transform: translateY(0); }Đ
    50% { transform: translateY(-2px); }Đ
    100% { transform: translateY(0); }Đ
}Đ
Đ
-----
```

```
@keyframes deltaPop {Đ
  0% { transform: scale(1); }Đ
  50% { transform: scale(1.3); }Đ
  100% { transform: scale(1); }Đ
}Đ
Đ
@keyframes pulse {Đ
  0%, 100% { box-shadow: 0 0 0 0 rgba(108,92,231,0.4); }Đ
  50% { box-shadow: 0 0 0 12px rgba(108,92,231,0); }Đ
}Đ
Đ
.btn.pulse {Đ
  animation: pulse 2s infinite;Đ
}Đ
Đ
.delta-result-card {Đ
  background: var(--surface2);Đ
  padding: 16px;Đ
  border-radius: 12px;Đ
  text-align: left;Đ
  line-height: 2;Đ
  border: 1px solid var(--border);Đ
}Đ
Đ
.delta-result-item {Đ
  display: flex;Đ
  align-items: center;Đ
  gap: 8px;Đ
  padding: 4px 0;Đ
  border-bottom: 1px solid var(--border);Đ
}Đ
.delta-result-item:last-child { border-bottom: none; }Đ
Đ
.delta-result-icon { font-size: 18px; }Đ
.delta-result-name { color: var(--text2); font-size: 13px; min-width: 40px;
}Đ
.delta-result-val { color: var(--accent2); font-weight: 700; margin-left: a
uto; }Đ
Đ
@media (max-width: 600px) {Đ
  .delta-grid {Đ
    grid-template-columns: repeat(2, 1fr);Đ
    gap: 8px;Đ
  }
}
```

```
}Đ
.delta-photo-icon { font-size: 28px; }Đ
.delta-display { font-size: 12px; }Đ
}Đ
Đ
/* === Wordle === */Đ
.wordle-grid { display: flex; flex-direction: column; gap: 6px; align-items
: center; }Đ
.wordle-row { display: flex; gap: 6px; }Đ
.wordle-cell {Đ
width: 52px; height: 52px;Đ
border: 2px solid var(--border);Đ
border-radius: 6px;Đ
display: flex; align-items: center; justify-content: center;Đ
font-size: 22px; font-weight: 700;Đ
transition: background 0.3s, transform 0.4s;Đ
text-transform: uppercase;Đ
}Đ
.wordle-key {Đ
flex: 1; min-width: 24px; padding: 8px 4px !important;Đ
font-size: 11px !important; font-weight: 600;Đ
background: var(--surface2) !important;Đ
color: var(--text) !important;Đ
border: 1px solid var(--border);Đ
}Đ
Đ
/* === Simon Says === */Đ
.simon-board {Đ
display: grid;Đ
grid-template-columns: 1fr 1fr;Đ
gap: 12px;Đ
max-width: 260px;Đ
margin: 0 auto;Đ
}Đ
Đ
/* === Whack-a-Mole === */Đ
.whack-grid {Đ
display: grid;Đ
grid-template-columns: repeat(3, 1fr);Đ
gap: 10px;Đ
max-width: 330px;Đ
margin: 0 auto;Đ
}Đ
.whack-hole {Đ
position: relative;Đ
aspect-ratio: 1;Đ
background: var(--surface2);Đ
```

=====

vĖfz\ n8b T -Æ^sSđ|û~ß V3.2

=====

{, 37 ~u

border-radius: 50%;Đ
cursor: pointer;Đ
overflow: hidden;Đ
display: flex;Đ

=====

vĖfz\ n8b T -Æ^sSđ|û~ß V3.2

=====

{, 62 ~u

align-items: center;Đ
justify-content: center;Đ
}Đ
.whack-mole {Đ
font-size: 36px;Đ
transform: translateY(60px);Đ
transition: transform 0.15s ease;Đ
z-index: 2;Đ
}Đ
.whack-mole.up { transform: translateY(0); }Đ
.whack-dirt {Đ
position: absolute; bottom: 0; left: 0; right: 0; height: 40%;Đ
background: #3d2e1a;Đ
border-radius: 50% 50% 0 0;Đ
}Đ
Đ
@media (max-width: 600px) {Đ
.wordle-cell { width: 40px; height: 40px; font-size: 18px; }Đ
}Đ
Đ
/* === D-Pad (Snake) === */Đ
.dpad {Đ
display: grid;Đ
grid-template-areas: ". up ." "left . right" ". down .";Đ
grid-template-columns: repeat(3, 56px);Đ
grid-template-rows: repeat(3, 56px);Đ
gap: 4px;Đ
justify-content: center;Đ
margin-top: 12px;Đ
}Đ
.dpad-btn {Đ
background: var(--surface2);Đ
color: var(--text);Đ
border: 2px solid var(--border);Đ
border-radius: 10px;Đ
font-size: 22px;Đ
cursor: pointer;Đ
user-select: none;Đ

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 38 ~u

```
-webkit-user-select: none;ð
touch-action: manipulation;ð
}ð
.dpad-btn:active { background: var(--accent); }ð
.dpad-up { grid-area: up; }ð
.dpad-down { grid-area: down; }ð
.dpad-left { grid-area: left; }ð
.dpad-right { grid-area: right; }ð
ð
/* === Tetris Controls === */ð
.tetris-controls {ð
  display: flex;ð
```

=====

vÊfz\ n8b T -Æ^sSð|û~ß V3.2

=====

{, 63 ~u

```
gap: 8px;ð
justify-content: center;ð
margin-top: 12px;ð
flex-wrap: wrap;ð
}ð
.tet-btn {ð
  flex: 1;ð
  min-width: 48px;ð
  padding: 12px 8px !important;ð
  font-size: 18px !important;ð
  touch-action: manipulation;ð
  user-select: none;ð
  -webkit-user-select: none;ð
}ð
ð
/* Hide D-pad & tetris controls on desktop, show on touch */ð
@media (hover: hover) and (pointer: fine) {ð
  .dpad { display: none; }ð
  .tetris-controls { display: none; }ð
}ð
```